Assignment 8: Hybrid SDN and Wi-Fi Network with a Mobile Station

Computer Networks Laboratory · Simulation using ns-3 (OpenFlow, Wi-Fi and Mobility modules)

Name	Ramnath Thakur	**Roll No.**	123CS0965
Simulator	ns-3.40	**Date**	12 October 2026

1. Aim

To simulate a hybrid wired/wireless SDN network made of three OpenFlow switches connected in a triangle, each with a wired host, and two Wi-Fi access points attached to two of the switches. A wireless station (STA) starts on one AP and moves towards the other with a `ConstantVelocityMobilityModel` while sending UDP traffic to a host on a different switch. The network is managed by a flooding controller. The goal is to observe how the STA's traffic is forwarded through the switches, how the controller installs and updates flows, and how OpenFlow copes with mobility, a mixed wired-wireless topology and a continuous UDP flow.

2. Software and Tools

Network simulator	ns-3.40 (C++), built with the `openflow`, `wifi`, `mobility`, `csma`, `bridge`, `internet`, `applications` and `flow-monitor` modules
OpenFlow library	NS-3 OpenFlow software switch library (OpenFlow 0.8.9), linked through `--with-openflow`
Wi-Fi standard	IEEE 802.11g, infrastructure mode, constant rate 24 Mbps (data) / 6 Mbps (control)
Compiler / OS	g++ 13.3, CMake + Ninja, Ubuntu 24.04
Post-processing	Python 3 with matplotlib (graphs from the CSV traces written by the simulation)

3. Theory

3.1 Software Defined Networking and OpenFlow

SDN separates the control plane from the data plane. The switches only hold flow tables; a logically central controller decides what goes into those tables. When a packet arrives that matches no flow entry, the switch buffers it and sends a `PACKET_IN` message to the controller. The controller replies with a `FLOW_MOD` that installs a match/action rule and, by giving the buffer id, also releases the waiting packet. Later packets of the same flow are handled in the switch at line rate without involving the controller.

3.2 Flooding controller

The simplest reactive controller. It does not learn MAC addresses or compute paths. For every table miss it installs an exact-match rule whose action is "send out of every port except the one it came in on". This guarantees delivery to the destination as long as the flooding graph is connected and loop-free. The cost is that every packet is copied to every link and host in the network. On a topology with a cycle (our triangle) plain flooding loops forever, so a spanning tree is needed; here the controller simply never floods over the S0–S1 link.

3.3 Wi-Fi infrastructure mode and handover

An AP broadcasts beacons roughly every 102.4 ms. A STA stays associated while it keeps hearing them. In ns-3, once `MaxMissedBeacons` consecutive beacons are lost the `StaWifiMac` declares the link lost, fires the `DeAssoc` trace, scans again, and associates with whichever AP with the same SSID it can hear (the `Assoc` trace). Each AP here is a bridge between its Wi-Fi interface and its Ethernet port, so from the switch's point of view the STA is just another MAC address that appears on the AP port.

3.4 ConstantVelocityMobilityModel

The node moves in a straight line from an initial position with a fixed velocity vector: $p(t) = p_0 + v \cdot t$. With $p_0 = (5, 0, 0)$ m and $v = (4, 0, 0)$ m/s the STA is at $x = 5 + 4t$.

4. Network Topology

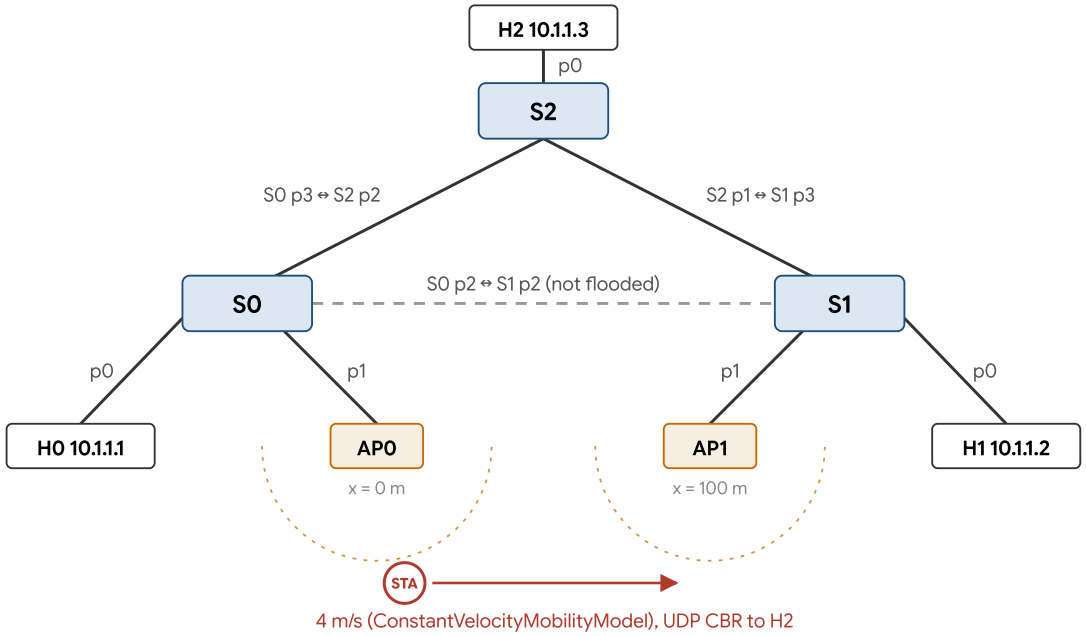


Figure 1. Simulated topology. Dotted circles show the 60 m radio range of each AP. The dashed S0–S1 link is kept out of the flooding tree.

Node	Role	Connected to / port	Address / position
S0, S1, S2	OpenFlow switch	Triangle S0–S1, S1–S2, S2–S0	No IP (layer 2 datapath)
H0, H1, H2	Wired host	Port 0 of S0, S1, S2	10.1.1.1, 10.1.1.2, 10.1.1.3
AP0	802.11g AP + bridge	Port 1 of S0	(0, 0) m, SSID `sdn-lab`
AP1	802.11g AP + bridge	Port 1 of S1	(100, 0) m, SSID `sdn-lab`
STA	Mobile station, UDP source	AP0, later AP1	10.1.1.4, starts at (5, 0) m

Simulation parameters

Wired links (all CSMA)	100 Mbps, 1 ms delay
Propagation loss	`RangePropagationLossModel`, MaxRange = 60 m (gives a sharp, repeatable cell edge)
Coverage overlap	40 m ≤ x ≤ 60 m (both APs audible)
STA mobility	`ConstantVelocityMobilityModel`, 4 m/s along +x
STA MAC	Passive scanning, `MaxMissedBeacons` = 5
Traffic	`UdpClient` on STA → `UdpServer` on H2, port 9000, 512 B every 10.24 ms (400 kbps CBR)
Traffic window	1.0 s to 29.0 s; simulation ends at 30 s
Controller	One `FloodingController` instance shared by S0, S1 and S2; flows are exact-match and permanent

5. Procedure

1. Built the OpenFlow switch library and configured ns-3.40 with `./ns3 configure --enable-modules="openflow;wifi;mobility;csma;bridge;internet;applications;flow-monitor" --with-openflow=<path>`.
2. Created 3 switch nodes, 3 hosts, 2 APs and 1 STA. Joined each host and each AP to its switch with a CSMA link, then created the three switch-to-switch links.
3. Installed an `OpenFlowSwitchNetDevice` on every switch node with `OpenFlowSwitchHelper`, passing the same controller object to all three.
4. Wrote `FloodingController` as a subclass of `ofi::Controller`. On `PACKET_IN` it extracts the flow key and installs a flow whose actions output to every port except the ingress port and the blocked S0–S1 port.
5. Installed 802.11g AP MACs on AP0/AP1 and a STA MAC on the STA (same SSID). Bridged each AP's Wi-Fi device with its CSMA device using `BridgeHelper`.
6. Gave the APs fixed positions and the STA a `ConstantVelocityMobilityModel`.
7. Assigned addresses from 10.1.1.0/24 to the hosts and the STA, started the UDP client and server.
8. Connected trace sinks to `StaWifiMac/Assoc`, `StaWifiMac/DeAssoc` and `UdpServer/Rx`, logged every `FLOW_MOD` from inside the controller and sampled each switch's flow table every 0.5 s through `GetChain()`. Also enabled FlowMonitor and a pcap on H2.
9. Ran `./ns3 run sdn-wifi-mobility` and plotted the CSV output.

6. Program

File: `scratch/sdn-wifi-mobility.cc`

```
/*
 * Assignment 8: Hybrid SDN + Wi-Fi network with a mobile station
 *
 *                         +---------+
 *              +----------|   S2    |----------+
 *              |          +---------+          |
 *              |               |               |
 *         +---------+         H2          +---------+
 *         |   S0    |=====(no-flood)======|   S1    |
 *         +---------+                     +---------+
 *           |     |                         |     |
 *          H0    AP0  ~ ~ ~ STA --> ~ ~ ~  AP1    H1
 *
 * S0, S1, S2 : OpenFlow 0.8.9 switches (ns-3 openflow module), triangle
 * H0..H2     : wired hosts, one per switch
 * AP0, AP1   : 802.11g access points, bridged onto S0 / S1
 * STA        : mobile station, ConstantVelocity, UDP CBR towards H2
 *
 * One controller (FloodingController) manages all three switches.
 */

#include "ns3/applications-module.h"
#include "ns3/bridge-module.h"
#include "ns3/core-module.h"
#include "ns3/csma-module.h"
#include "ns3/flow-monitor-module.h"
#include "ns3/internet-module.h"
#include "ns3/mobility-module.h"
#include "ns3/network-module.h"
#include "ns3/openflow-module.h"
#include "ns3/wifi-module.h"

#include <fstream>
#include <iomanip>
#include <map>
#include <set>

using namespace ns3;

NS_LOG_COMPONENT_DEFINE("SdnWifiMobility");

static std::ofstream g_flowLog;   // every flow_mod the controller pushes
static std::ofstream g_eventLog;  // association / handover events
static std::ofstream g_rxLog;     // per-interval goodput at H2
static std::ofstream g_tableLog;  // flow-table occupancy per switch over time

static std::map<Ptr<OpenFlowSwitchNetDevice>, std::string> g_swName;

/* ---------------------------------------------------------------------- */
/* Flooding controller                                                    */
/* ---------------------------------------------------------------------- */
// On every PACKET_IN it installs an exact-match flow that floods the packet
// and releases the buffered packet through that flow. It keeps no MAC table,
// so it never has to "re-learn" a station after a handover.
//
// Two details matter on a triangle:
//  1. Plain flooding loops forever, so the controller keeps a spanning tree by
//     never flooding over the S0<->S1 link (both ends are marked blocked).
//  2. ns-3's switch compares key.flow.in_port (network byte order, set by
//     flow_extract) with a host-order port index when it expands OFPP_FLOOD,
//     so the ingress port is not skipped for any port >= 1. To avoid that, the
//     controller expands the flood itself into one OFPAT_OUTPUT per port.
namespace ns3
{
```

```
namespace ofi
{
class FloodingController : public Controller
{
  public:
    static TypeId GetTypeId()
    {
        static TypeId tid = TypeId("ns3::ofi::FloodingController")
                                .SetParent<Controller>()
                                .SetGroupName("Openflow")
                                .AddConstructor<FloodingController>();
        return tid;
    }

    void ReceiveFromSwitch(Ptr<OpenFlowSwitchNetDevice> swtch, ofpbuf* buffer) override
    {
        if (m_switches.find(swtch) == m_switches.end())
        {
            return;
        }
        uint8_t type = GetPacketType(buffer);
        if (type == OFPT_PACKET_IN)
        {
            auto opi = (ofp_packet_in*)ofpbuf_try_pull(buffer, offsetof(ofp_packet_in, data));
            int port = ntohs(opi->in_port);

            sw_flow_key key;
            key.wildcards = 0;
            flow_extract(buffer, port != -1 ? port : OFPP_NONE, &key.flow);

            uint16_t inPort = ntohs(key.flow.in_port);
            std::vector<ofp_action_output> x;
            std::ostringstream outs;
            for (uint32_t p = 0; p < swtch->GetNSwitchPorts(); p++)
            {
                if (p == inPort || m_blocked[swtch].count(p))
                {
                    continue;
                }
                ofp_action_output a;
                memset(&a, 0, sizeof(a));
                a.type = htons(OFPAT_OUTPUT);
                a.len = htons(sizeof(ofp_action_output));
                a.port = p;
                x.push_back(a);
                outs << (outs.tellp() ? " " : "") << p;
            }

            ofp_flow_mod* ofm = BuildFlow(key,
                                          opi->buffer_id,
                                          OFPFC_ADD,
                                          x.data(),
                                          x.size() * sizeof(ofp_action_output),
                                          OFP_FLOW_PERMANENT,
                                          OFP_FLOW_PERMANENT);
            SendToSwitch(swtch, ofm, ofm->header.length);
            m_installed[swtch]++;
            LogFlow(swtch, key, outs.str());
        }
        else if (type == OFPT_FLOW_EXPIRED)
        {
            g_flowLog << std::fixed << std::setprecision(3) << Simulator::Now().GetSeconds()
                      << "," << g_swName[swtch] << ",EXPIRED,,,,,,," << std::endl;
        }
    }

    // Exclude a port from flooding (spanning-tree style).
    void BlockFlood(Ptr<OpenFlowSwitchNetDevice> swtch, uint32_t portNo)
    {
        m_blocked[swtch].insert(portNo);
```

```
        NS_LOG_INFO(g_swName[swtch] << " port " << portNo << " excluded from flooding");
    }

    uint32_t Installed(Ptr<OpenFlowSwitchNetDevice> s)
    {
        return m_installed[s];
    }

  private:
    void LogFlow(Ptr<OpenFlowSwitchNetDevice> swtch, const sw_flow_key& key, std::string outs)
    {
        Mac48Address src;
        Mac48Address dst;
        src.CopyFrom(key.flow.dl_src);
        dst.CopyFrom(key.flow.dl_dst);
        uint16_t ethType = ntohs(key.flow.dl_type);
        std::string kind = "OTHER";
        std::ostringstream nw;
        if (ethType == 0x0806)
        {
            kind = "ARP";
        }
        else if (ethType == 0x0800)
        {
            kind = key.flow.nw_proto == 17 ? "UDP" : "IPv4";
        }
        if (ethType == 0x0800 || ethType == 0x0806)
        {
            nw << Ipv4Address(ntohl(key.flow.nw_src)) << ","
               << Ipv4Address(ntohl(key.flow.nw_dst));
        }
        else
        {
            nw << ",";
        }
        g_flowLog << std::fixed << std::setprecision(3) << Simulator::Now().GetSeconds() << ","
                  << g_swName[swtch] << ",ADD," << ntohs(key.flow.in_port) << "," << kind << ","
                  << src << "," << dst << "," << nw.str() << ",out:" << outs << std::endl;
    }

    std::map<Ptr<OpenFlowSwitchNetDevice>, uint32_t> m_installed;
    std::map<Ptr<OpenFlowSwitchNetDevice>, std::set<uint32_t>> m_blocked;
};

NS_OBJECT_ENSURE_REGISTERED(FloodingController);
} // namespace ofi
} // namespace ns3

/* ------------------------------------------------------------------- */
/* Tracing helpers                                                     */
/* ------------------------------------------------------------------- */
static std::map<Mac48Address, std::string> g_apName;
static uint64_t g_rxBytes = 0;
static uint64_t g_rxPkts = 0;
static const double kSample = 0.5;

static void
OnAssoc(Mac48Address bssid)
{
    g_eventLog << std::fixed << std::setprecision(3) << Simulator::Now().GetSeconds()
               << ",ASSOC," << g_apName[bssid] << std::endl;
    NS_LOG_UNCOND(Simulator::Now().GetSeconds() << "s STA associated with " << g_apName[bssid]);
}

static void
OnDeassoc(Mac48Address bssid)
{
    g_eventLog << std::fixed << std::setprecision(3) << Simulator::Now().GetSeconds()
               << ",DEASSOC," << g_apName[bssid] << std::endl;
    NS_LOG_UNCOND(Simulator::Now().GetSeconds() << "s STA lost " << g_apName[bssid]);
```

```
}

static void
OnRx(Ptr<const Packet> p)
{
    g_rxBytes += p->GetSize();
    g_rxPkts++;
}

static void
SampleRx(Ptr<MobilityModel> sta)
{
    double kbps = g_rxBytes * 8.0 / kSample / 1000.0;
    g_rxLog << std::fixed << std::setprecision(2) << Simulator::Now().GetSeconds() << "," << kbps
            << "," << g_rxPkts << "," << sta->GetPosition().x << std::endl;
    g_rxBytes = 0;
    g_rxPkts = 0;
    Simulator::Schedule(Seconds(kSample), &SampleRx, sta);
}

static void
SampleTables(std::vector<Ptr<OpenFlowSwitchNetDevice>> sws)
{
    g_tableLog << std::fixed << std::setprecision(1) << Simulator::Now().GetSeconds();
    for (auto& s : sws)
    {
        sw_chain* c = s->GetChain();
        unsigned int n = 0;
        unsigned long lookups = 0;
        unsigned long hits = 0;
        for (int i = 0; i < c->n_tables; i++)
        {
            sw_table_stats st;
            c->tables[i]->stats(c->tables[i], &st);
            n += st.n_flows;
            lookups += st.n_lookup;
            hits += st.n_matched;
        }
        g_tableLog << "," << n << "," << lookups << "," << hits;
    }
    g_tableLog << std::endl;
    Simulator::Schedule(Seconds(0.5), &SampleTables, sws);
}

int
main(int argc, char* argv[])
{
    double simTime = 30.0;
    double speed = 4.0;       // m/s along +x
    double apGap = 100.0;     // distance AP0 -> AP1
    double range = 60.0;      // radio range of each AP
    std::string rate = "400kbps";
    uint32_t pktSize = 512;
    std::string out = "results";

    CommandLine cmd(__FILE__);
    cmd.AddValue("simTime", "Simulation time (s)", simTime);
    cmd.AddValue("speed", "STA speed (m/s)", speed);
    cmd.AddValue("range", "AP radio range (m)", range);
    cmd.AddValue("out", "Output prefix", out);
    cmd.Parse(argc, argv);

    LogComponentEnable("SdnWifiMobility", LOG_LEVEL_INFO);

    g_flowLog.open(out + "-flows.csv");
    g_flowLog << "time,switch,op,in_port,type,dl_src,dl_dst,nw_src,nw_dst,action" << std::endl;
    g_eventLog.open(out + "-events.csv");
    g_eventLog << "time,event,ap" << std::endl;
    g_rxLog.open(out + "-rx.csv");
    g_rxLog << "time,kbps,pkts,sta_x" << std::endl;
```

```
    g_tableLog.open(out + "-tables.csv");
    g_tableLog << "time,S0_flows,S0_lookups,S0_hits,S1_flows,S1_lookups,S1_hits,S2_flows,"
                  "S2_lookups,S2_hits"
               << std::endl;

    /* ---------------- nodes ---------------- */
    NodeContainer switches;
    switches.Create(3);
    NodeContainer hosts;
    hosts.Create(3);
    NodeContainer aps;
    aps.Create(2);
    NodeContainer sta;
    sta.Create(1);

    CsmaHelper csma;
    csma.SetChannelAttribute("DataRate", DataRateValue(DataRate("100Mbps")));
    csma.SetChannelAttribute("Delay", TimeValue(MilliSeconds(1)));

    std::vector<NetDeviceContainer> swPorts(3);
    NetDeviceContainer hostDevs;
    NetDeviceContainer apWired;

    // host <-> switch   (port 0 on every switch)
    for (uint32_t i = 0; i < 3; i++)
    {
        NetDeviceContainer l = csma.Install(NodeContainer(hosts.Get(i), switches.Get(i)));
        hostDevs.Add(l.Get(0));
        swPorts[i].Add(l.Get(1));
    }
    // AP <-> switch     (port 1 on S0 and S1)
    for (uint32_t i = 0; i < 2; i++)
    {
        NetDeviceContainer l = csma.Install(NodeContainer(aps.Get(i), switches.Get(i)));
        apWired.Add(l.Get(0));
        swPorts[i].Add(l.Get(1));
    }
    // triangle: S0-S1, S1-S2, S2-S0
    uint32_t pairs[3][2] = {{0, 1}, {1, 2}, {2, 0}};
    std::map<std::pair<uint32_t, uint32_t>, uint32_t> portOf; // (sw, peer) -> port index
    for (auto& pr : pairs)
    {
        NetDeviceContainer l = csma.Install(NodeContainer(switches.Get(pr[0]), switches.Get(pr[1])));
        portOf[{pr[0], pr[1]}] = swPorts[pr[0]].GetN();
        swPorts[pr[0]].Add(l.Get(0));
        portOf[{pr[1], pr[0]}] = swPorts[pr[1]].GetN();
        swPorts[pr[1]].Add(l.Get(1));
    }

    /* ---------------- OpenFlow ---------------- */
    Ptr<ofi::FloodingController> ctrl = CreateObject<ofi::FloodingController>();
    OpenFlowSwitchHelper ofHelper;
    std::vector<Ptr<OpenFlowSwitchNetDevice>> ofDevs;
    for (uint32_t i = 0; i < 3; i++)
    {
        NetDeviceContainer d = ofHelper.Install(switches.Get(i), swPorts[i], ctrl);
        Ptr<OpenFlowSwitchNetDevice> of = DynamicCast<OpenFlowSwitchNetDevice>(d.Get(0));
        g_swName[of] = "S" + std::to_string(i);
        ofDevs.push_back(of);
    }
    // break the loop: S0-S1 link is excluded from flooding on both ends
    ctrl->BlockFlood(ofDevs[0], portOf[{0, 1}]);
    ctrl->BlockFlood(ofDevs[1], portOf[{1, 0}]);

    /* ---------------- Wi-Fi ---------------- */
    YansWifiChannelHelper chan;
    chan.SetPropagationDelay("ns3::ConstantSpeedPropagationDelayModel");
    chan.AddPropagationLoss("ns3::RangePropagationLossModel", "MaxRange", DoubleValue(range));
    YansWifiPhyHelper phy;
    phy.SetChannel(chan.Create());
```

```
WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211g);
wifi.SetRemoteStationManager("ns3::ConstantRateWifiManager",
                             "DataMode",
                             StringValue("ErpOfdmRate24Mbps"),
                             "ControlMode",
                             StringValue("ErpOfdmRate6Mbps"));
WifiMacHelper mac;
Ssid ssid("sdn-lab");

mac.SetType("ns3::ApWifiMac", "Ssid", SsidValue(ssid));
NetDeviceContainer apWifi = wifi.Install(phy, mac, aps);

mac.SetType("ns3::StaWifiMac",
            "Ssid",
            SsidValue(ssid),
            "ActiveProbing",
            BooleanValue(false),
            "MaxMissedBeacons",
            UintegerValue(5));
NetDeviceContainer staWifi = wifi.Install(phy, mac, sta);

// each AP bridges its wireless side onto the wired port towards its switch
BridgeHelper bridge;
for (uint32_t i = 0; i < 2; i++)
{
    NetDeviceContainer b;
    b.Add(apWifi.Get(i));
    b.Add(apWired.Get(i));
    bridge.Install(aps.Get(i), b);
    g_apName[Mac48Address::ConvertFrom(apWifi.Get(i)->GetAddress())] = "AP" + std::to_string(i);
}

/* ---------------- mobility ---------------- */
MobilityHelper fixed;
Ptr<ListPositionAllocator> pos = CreateObject<ListPositionAllocator>();
pos->Add(Vector(0, 0, 0));     // AP0
pos->Add(Vector(apGap, 0, 0)); // AP1
fixed.SetPositionAllocator(pos);
fixed.SetMobilityModel("ns3::ConstantPositionMobilityModel");
fixed.Install(aps);

MobilityHelper moving;
moving.SetMobilityModel("ns3::ConstantVelocityMobilityModel");
moving.Install(sta);
Ptr<ConstantVelocityMobilityModel> staMob = sta.Get(0)->GetObject<ConstantVelocityMobilityModel>();
staMob->SetPosition(Vector(5, 0, 0));
staMob->SetVelocity(Vector(speed, 0, 0));

/* ---------------- IP ---------------- */
InternetStackHelper stack;
stack.Install(hosts);
stack.Install(sta);
Ipv4AddressHelper ip;
ip.SetBase("10.1.1.0", "255.255.255.0");
Ipv4InterfaceContainer hostIf = ip.Assign(hostDevs); // 10.1.1.1 .. .3
Ipv4InterfaceContainer staIf = ip.Assign(staWifi);   // 10.1.1.4

/* ---------------- traffic: STA -> H2 (UDP CBR) ---------------- */
uint16_t port = 9000;
UdpServerHelper server(port);
ApplicationContainer srv = server.Install(hosts.Get(2));
srv.Start(Seconds(0.5));
srv.Stop(Seconds(simTime));

UdpClientHelper client(hostIf.GetAddress(2), port);
double interval = pktSize * 8.0 / DataRate(rate).GetBitRate();
client.SetAttribute("Interval", TimeValue(Seconds(interval)));
client.SetAttribute("PacketSize", UintegerValue(pktSize));
```

```
    client.SetAttribute("MaxPackets", UintegerValue(1000000));
    ApplicationContainer cli = client.Install(sta.Get(0));
    cli.Start(Seconds(1.0));
    cli.Stop(Seconds(simTime - 1));

    srv.Get(0)->TraceConnectWithoutContext("Rx", MakeCallback(&OnRx));

    /* ---------------- traces ---------------- */

Config::ConnectWithoutContext("/NodeList/*/DeviceList/*/$ns3::WifiNetDevice/Mac/$ns3::StaWifiMac/Assoc",
                              MakeCallback(&OnAssoc));

Config::ConnectWithoutContext("/NodeList/*/DeviceList/*/$ns3::WifiNetDevice/Mac/$ns3::StaWifiMac/DeAssoc",
                              MakeCallback(&OnDeassoc));
    Simulator::Schedule(Seconds(kSample), &SampleRx, staMob);
    Simulator::Schedule(Seconds(0.5), &SampleTables, ofDevs);

    csma.EnablePcap(out + "-h2", hostDevs.Get(2), true);

    FlowMonitorHelper fmh;
    Ptr<FlowMonitor> fm = fmh.InstallAll();

    Simulator::Stop(Seconds(simTime));
    Simulator::Run();

    /* ---------------- summary ---------------- */
    Ptr<UdpServer> us = DynamicCast<UdpServer>(srv.Get(0));
    Ptr<UdpClient> uc = DynamicCast<UdpClient>(cli.Get(0));
    std::cout << "\n=== UDP summary ===\n"
              << "Received at H2      : " << us->GetReceived() << "\n"
              << "Lost (seq gaps)     : " << us->GetLost() << "\n";

    fm->CheckForLostPackets();
    auto cls = DynamicCast<Ipv4FlowClassifier>(fmh.GetClassifier());
    std::ofstream fmOut(out + "-flowmon.csv");
    fmOut << "flow,src,dst,proto,tx,rx,lost,throughput_kbps,mean_delay_ms,mean_jitter_ms\n";
    for (auto& kv : fm->GetFlowStats())
    {
        auto t = cls->FindFlow(kv.first);
        auto& s = kv.second;
        double dur = (s.timeLastRxPacket - s.timeFirstTxPacket).GetSeconds();
        double thr = dur > 0 ? s.rxBytes * 8.0 / dur / 1000.0 : 0;
        double d = s.rxPackets ? s.delaySum.GetSeconds() * 1000 / s.rxPackets : 0;
        double j = s.rxPackets > 1 ? s.jitterSum.GetSeconds() * 1000 / (s.rxPackets - 1) : 0;
        fmOut << kv.first << "," << t.sourceAddress << "," << t.destinationAddress << ","
              << (int)t.protocol << "," << s.txPackets << "," << s.rxPackets << ","
              << s.txPackets - s.rxPackets << "," << thr << "," << d << "," << j << "\n";
        std::cout << "Flow " << kv.first << " " << t.sourceAddress << " -> "
                  << t.destinationAddress << "  tx=" << s.txPackets << " rx=" << s.rxPackets
                  << " thr=" << thr << " kbps delay=" << d << " ms\n";
    }
    for (uint32_t i = 0; i < 3; i++)
    {
        std::cout << "S" << i << " flow_mods installed: " << ctrl->Installed(ofDevs[i]) << "\n";
    }

    Simulator::Destroy();
    return 0;
}
```

7. Output

7.1 Console output

```
$ ./ns3 run sdn-wifi-mobility
S0 port 2 excluded from flooding
S1 port 2 excluded from flooding
```

```
0.12161s STA associated with AP0
14.222s STA lost AP0
14.3436s STA associated with AP1

=== UDP summary ===
Received at H2      : 2677
Lost (seq gaps)     : 58
Flow 1 10.1.1.4 -> 10.1.1.3  tx=2735 rx=2677 thr=413.03 kbps delay=3.39491 ms
S0 flow_mods installed: 5
S1 flow_mods installed: 5
S2 flow_mods installed: 5
```

7.2 Flows installed by the controller

Every row is one `FLOW_MOD` sent by the controller in response to a `PACKET_IN`. MAC `..:13` is the STA, `..:05` is H2. Port numbers are the switch's own port indices (see Figure 1).

Time (s)	Switch	In port	Type	dl_src → dl_dst	nw_src → nw_dst	Actions
1.010	S0	1	ARP	..:13 → ff:ff:ff:ff:ff:ff	-	output 0 3
1.011	S2	2	ARP	..:13 → ff:ff:ff:ff:ff:ff	-	output 0 1
1.012	S1	3	ARP	..:13 → ff:ff:ff:ff:ff:ff	-	output 0 1
1.013	S2	0	ARP	..:05 → ..:13	-	output 1 2
1.014	S1	3	ARP	..:05 → ..:13	-	output 0 1
1.014	S0	3	ARP	..:05 → ..:13	-	output 0 1
1.017	S0	1	UDP	..:13 → ..:05	10.1.1.4 → 10.1.1.3	output 0 3
1.018	S2	2	UDP	..:13 → ..:05	10.1.1.4 → 10.1.1.3	output 0 1
1.019	S1	3	UDP	..:13 → ..:05	10.1.1.4 → 10.1.1.3	output 0 1
14.356	S1	1	ARP	..:13 → ff:ff:ff:ff:ff:ff	-	output 0 3
14.357	S2	1	ARP	..:13 → ff:ff:ff:ff:ff:ff	-	output 0 2
14.358	S0	3	ARP	..:13 → ff:ff:ff:ff:ff:ff	-	output 0 1
14.362	S1	1	UDP	..:13 → ..:05	10.1.1.4 → 10.1.1.3	output 0 3
14.363	S2	1	UDP	..:13 → ..:05	10.1.1.4 → 10.1.1.3	output 0 2
14.364	S0	3	UDP	..:13 → ..:05	10.1.1.4 → 10.1.1.3	output 0 1

Table 1. Complete controller log for the run. Nothing else reached the controller.

7.3 Flow table occupancy

t (s)	S0 flows	S0 hits / lookups	S1 flows	S1 hits / lookups	S2 flows	S2 hits / lookups
2.0	3	97 / 103	3	97 / 103	3	97 / 103
8.0	3	683 / 689	3	683 / 689	3	683 / 689
13.5	3	1220 / 1226	3	1220 / 1226	3	1220 / 1226
14.0	3	1245 / 1251	3	1245 / 1251	3	1245 / 1251
14.5	5	1260 / 1270	5	1260 / 1270	5	1260 / 1270
20.0	5	1797 / 1807	5	1797 / 1807	5	1797 / 1807
29.5	5	2676 / 2686	5	2676 / 2686	5	2676 / 2686

Table 2. Flow entries, table hits and lookups sampled from each switch's chain.

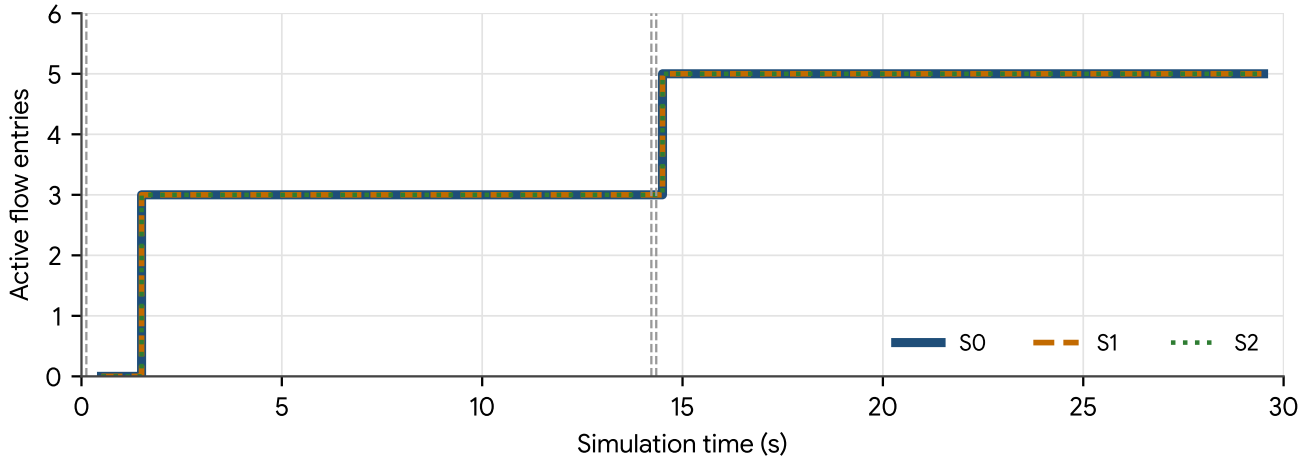


Figure 2. Active flow entries per switch. All three curves overlap because every switch learns the same set of flows (see Section 8.3). Dashed lines mark the handover events.

7.4 UDP goodput at H2 and handover

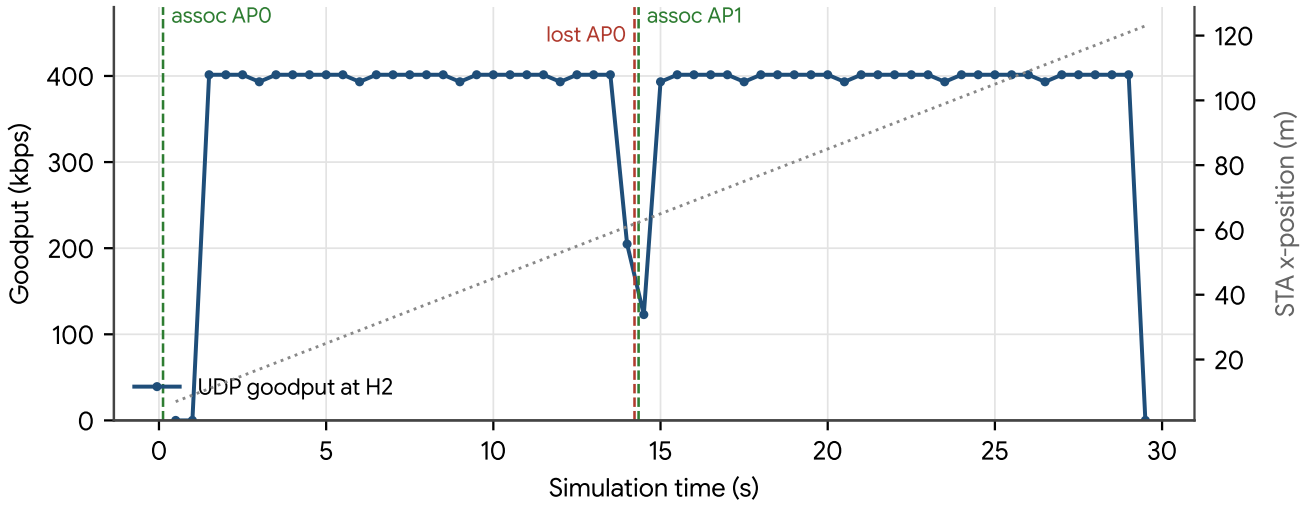


Figure 3. Goodput received at H2 in 0.5 s bins (left axis) with the STA's x-position (dotted, right axis).

Metric (FlowMonitor, 10.1.1.4 → 10.1.1.3, UDP)	Value
Packets transmitted by STA	2735
Packets received at H2	2677
Packets lost	58 (2.12 %)
Average throughput	413.0 kbps
Mean end-to-end delay	3.39 ms
Mean jitter	11.5 µs

Table 3. End-to-end statistics for the UDP flow.

8. Observations and Analysis

8.1 Initial flow setup (t = 1.0 s)

The first thing the STA sends is not UDP but an ARP request for 10.1.1.3. It reaches S0 on port 1 (AP0). S0 has no matching entry, so it raises a `PACKET_IN` and the controller installs *ARP, in_port 1 → output 0 3*. Port 2

(towards S1) is skipped because it is the blocked edge. The request then arrives at S2 on port 2 and at S1 on port 3, and each of those switches goes through the same miss / install cycle. H2 answers, and the reply generates one more flow per switch. Only after that does the first UDP datagram leave the STA and install three more flows (S0 → S2 → S1). The whole exchange, from the first ARP miss to the last UDP flow, takes 9 ms. From then on the controller is silent: in Table 2 the gap between lookups and hits stays fixed while the lookup count climbs into the thousands, so every later packet is matched inside the switch.

8.2 Behaviour during mobility

- **t = 0.12 s:** STA associates with AP0 (only AP0 is within 60 m).
- **t = 8.75 s:** STA crosses x = 40 m and can hear AP1 as well, but nothing happens. A station does not roam just because a second AP is audible; it stays with AP0.
- **t = 13.75 s:** STA crosses x = 60 m and is out of AP0's range. Its frames are no longer acknowledged and AP0's beacons stop arriving.
- **t = 14.222 s:** after 5 missed beacons the MAC declares the link lost (`DeAssoc AP0`).
- **t = 14.344 s:** it hears a beacon from AP1 and associates (`Assoc AP1`).
- **t = 14.356 s:** a new ARP request appears. ns-3 registers `ArpCache::Flush` as a link-change callback, so the cache is cleared when the Wi-Fi link goes down and up, so the STA has to resolve H2 again.
- **t = 14.362–14.364 s:** UDP now enters the SDN domain at S1 port 1 instead of S0 port 1. That is a new `in_port`, so the existing flows do not match and S1, S2 and S0 each raise one `PACKET_IN`. The controller installs *in_port 1 → output 0 3* on S1, *in_port 1 → output 0 2* on S2, and S0 gets a flow for the packets now arriving from S2.

The total outage is roughly 13.75 s to 14.36 s, about 0.6 s, which at 97.7 packets per second matches the 58 lost packets reported by both FlowMonitor and the UDP sequence numbers. The dip is visible in Figure 3 in the 14.0 and 14.5 s bins. The delay added by the controller itself (three new flow setups) is only a few milliseconds; nearly all of the loss is the Wi-Fi layer detecting the lost AP. No packets were lost anywhere in the wired SDN part.

8.3 How OpenFlow handled the mobility

Because the controller floods instead of learning, it never needs to be told that the STA moved. There is no stale "STA is behind port X" entry to fix, so delivery recovers as soon as the STA is reachable on Wi-Fi again. The flip side shows up in Table 2: S0, S1 and S2 have identical lookup counts. Every UDP packet is copied over the whole spanning tree and also delivered to H0 and H1, which do not want it. With one 400 kbps flow this is harmless, but the load on every link grows with the total traffic in the network, not with the traffic on that path.

Old flows are never removed. After the handover S0 still holds the entries for traffic arriving from AP0, and the table ends with 5 entries per switch instead of 3. The flows were installed as permanent on purpose: the ns-3 OpenFlow library stamps flows with the host's wall-clock time (`time_now()`), not simulation time, so idle and hard timeouts would not follow the simulated clock in a run that finishes in under a second of real time. A learning or path-computing controller would need explicit `OFPFC_DELETE` or `OFPFC_MODIFY` messages on handover.

8.4 Hybrid wired/wireless forwarding

The APs are plain layer-2 bridges, so the switches see the STA's own MAC address (`..:13`) on the AP port. The OpenFlow flow key treats a wireless host like any wired host; the only clue that a host is wireless is that its `in_port` changes at handover. Delay is 3.4 ms end to end, which is dominated by the three 1 ms CSMA hops plus Wi-Fi channel access.

Problem found during the experiment. My first version used the standard `OFPP_FLOOD` action together with `OFPPC_NO_FLOOD` on the S0–S1 ports. the UDP server on H2 counted 17 257 received datagrams for 2 735 sent and the mean delay rose to 395 ms. The cause is in ns-3's switch: `flow_extract()` stores `in_port` in network byte order, while `ExecuteActions()` passes it to the flood routine without `ntohs()`. For any port number above 0 the ingress port is therefore not excluded and each flooded frame is sent straight back where it came from, creating a loop even with a spanning tree. The final controller avoids this by expanding the flood itself into one `OUTPUT` action per allowed port, which is why Table 1 lists explicit port lists instead of `FLOOD`.

9. Result

The hybrid SDN and Wi-Fi network was simulated successfully in ns-3. Traffic from the mobile STA reached H2 through the OpenFlow switches before and after the handover, first on the path AP0 → S0 → S2 and then on AP1 → S1 → S2. The controller installed 15 flow entries in total, all of them reactively and all within a few milliseconds of the first packet of each new flow. 2677 of 2735 UDP packets were delivered (97.88 %). All 58 losses occurred during the 0.6 s Wi-Fi handover, none inside the SDN switches.

10. Conclusion

A flooding controller handles mobility well because it keeps no location state: once the STA reassociates, its packets enter the network on a new port, cause one table miss per switch and are forwarded again without any special handover logic. The price is that every packet is sent over the whole spanning tree, stale flow entries stay in the tables, and a triangle (or any cycle) needs loop protection. In this setup the handover delay came from the Wi-Fi layer (beacon loss detection plus reassociation, about 0.6 s), not from SDN. The run also exposed a byte-order bug in the ns-3 OpenFlow flood action, which turned the triangle into a forwarding loop until the controller was changed to list output ports explicitly.

Student's signature

Faculty's signature